Especificación de Requisitos de Software — Sonido Vivo

Proyecto Semestral · DSY1104 Desarrollo Full Stack II · Duoc UC Caso **Forma B — Tienda Sonido Vivo**

Versión	**2.0** — etapa **Evaluación Parcial 2** (versión entregada)
Fecha	20 de octubre de 2026
Autora	Sol León
Sección	005D
Repositorio	https://github.com/s0lci700/SonidoVivo

> Este documento se construye de forma progresiva durante el semestre y describe el sistema **tal como está construido**, no una propuesta ideal. Cada requisito indica su estado real en la versión entregada para la EP2.

---

1. Introducción

1.1 Propósito

Especificar los requisitos de **Sonido Vivo**, la tienda en línea de una tienda de instrumentos y equipos musicales de Viña del Mar. El documento sirve como referencia para el desarrollo, como evidencia de la Evaluación Parcial 2 y como base para las etapas siguientes del proyecto semestral.

1.2 Problema que resuelve

Sonido Vivo vende principalmente de forma presencial, pero recibe cada vez más pedidos por WhatsApp e Instagram desde otras ciudades. Según el documento del caso (§3.3):

- 3 de cada 10 mensajes de clientes remotos quedan sin respuesta el mismo día.
- No existe un catálogo actualizado que el cliente pueda consultar por su cuenta.
- El stock vive en un Excel que puede estar desactualizado durante horas, lo que provoca ventas de productos sin existencia.
- No hay registro centralizado de pedidos ni historial de compras por cliente.

1.3 Alcance del producto

Sonido Vivo permite a un cliente revisar el catálogo con precios y disponibilidad, armar un carrito y confirmar un pedido con despacho o retiro en tienda; y permite al personal mantener el catálogo y el stock desde un panel de administración.

El sistema es **un solo producto que evoluciona por etapas**; el repositorio no se reinicia en cada evaluación:

Etapa	Tecnología	Estado
Parcial 2	React 19 + Vite, React Router, Tailwind CSS 4, datos simulados en JSON + `localStorage`, Vitest + React Testing Library	**entregada**
Parcial 3	Microservicios Spring Boot, MySQL, API REST, JWT, mapa	pendiente
Examen	Integración, despliegue en AWS con Docker y cierre	pendiente

1.4 Alcance de esta versión (Parcial 2)

Esta etapa es **exclusivamente de front-end**. Cada requisito se clasifica en:

- **En alcance P2**: se construye en esta etapa con datos simulados.
- **Diferido**: depende del backend o quedó fuera del alcance de esta entrega; corresponde al Parcial 3 o al examen.

Las vistas del Anexo 1 (figuras 1 a 10) son, según el propio documento, **sugerencias**. Este proyecto implementa el flujo de compra completo y el mantenimiento de productos; las vistas sugeridas que no se construyeron están registradas como diferidas en la sección 3.

1.5 Definiciones

Término	Significado
Catálogo	Conjunto de productos; parte de `frontend/src/data/productos.json` y se mantiene en `localStorage`
Carrito	Selección temporal de productos con cantidades
Pedido	Compra confirmada por el cliente, con despacho o retiro en tienda
Componente	Función de React que recibe `props` y devuelve interfaz
Capa de servicios	Módulos de `frontend/src/services/` por los que pasa todo acceso a datos
SPA	Aplicación de una sola página; la navegación la resuelve React Router en el navegador
`localStorage`	Almacenamiento clave-valor del navegador, persistente entre sesiones
CRUD	Crear, leer, actualizar y eliminar

Término	Significado
CLP	Peso chileno
ERS	Especificación de Requisitos de Software (este documento)

1.6 Referencias

1. `documentacion/instrucciones/DSY1104 – Forma B – Tienda Sonido Vivo.docx`: documento del caso.
2. `documentacion/instrucciones/DSY1104 – Forma B – Catalogo Sonido Vivo.xlsx`: catálogo de productos.
3. `documentacion/instrucciones/EA2/Semana_09_Evaluacion_Parcial_2/`: rúbrica oficial, Anexo 1 y guías de la EP2.
4. `documentacion/cobertura-testing-EP2.md`: documento de cobertura del testing.
5. `documentacion/defensa-EP2/`: guía técnica del código (recorridos, pruebas, decisiones).
6. `README.md`: instrucciones de ejecución.
7. `DESIGN.md` y `PRODUCT.md`: sistema visual y contexto de diseño.

---

2. Descripción general

2.1 Perspectiva del producto

En el Parcial 2 el sistema es una SPA de React sin servidor de aplicación ni base de datos. Las categorías, regiones y usuarios se leen de archivos JSON locales; el catálogo parte de `productos.json` y se guarda en `localStorage` para que el CRUD de administración persista; la sesión se simula en `localStorage`.

Esta arquitectura es **deliberadamente temporal**. Los componentes nunca leen `data/` directamente: lo hacen a través de `services/`, con funciones `async`. En el Parcial 3 solo cambia esa capa, que pasa a usar `fetch` contra la API REST manteniendo los mismos nombres y firmas.

2.2 Funciones principales

- Navegación entre vistas con encabezado y pie de página comunes.
- Catálogo con precio, disponibilidad y filtro por categoría.
- Ficha individual de producto.
- Carrito con cantidades limitadas por el stock.
- Checkout con validaciones, despacho o retiro, y resultado de la compra.

- Inicio y cierre de sesión simulados.
- Panel de administración con indicadores y CRUD de productos.
- Páginas institucionales: Inicio, Nosotros y Contacto.

2.3 Roles de usuario

El documento del caso (§0.2) exige tres roles mínimos.

Rol	Descripción	En el Parcial 2
Cliente	Ve el catálogo, arma el carrito, hace pedidos y ve su historial	Catálogo, carrito y compra simulada, sin iniciar sesión
Vendedor	Actualiza stock y estado de los pedidos	Existe como usuario simulado; sus vistas se difieren al P3
Administrador	Acceso total; gestiona productos, usuarios y roles	Panel y CRUD de productos

2.4 Restricciones

- **R-01**: el frontend se construye íntegramente con React (caso §0.1).
- **R-02**: la interfaz funciona en 360, 768 y 1280 px; en móvil la navegación se colapsa en un menú tipo hamburguesa (caso §0.3).
- **R-03**: un solo framework de CSS. La pauta impresa nombra Bootstrap; el curso autorizó Bootstrap o Tailwind CSS (`00_Mensaje_AVA_EP2.txt`, `03_Orientaciones_Generales`). Este proyecto usa **Tailwind CSS v4**.
- **R-04**: las pruebas se escriben con **Vitest + React Testing Library**, la actualización técnica indicada por el curso en lugar de Jasmine/Karma.
- **R-05**: la autenticación del Parcial 2 es una **simulación** en `localStorage`. No constituye seguridad real y así se presenta.
- **R-06**: no se elimina el trabajo de una etapa para empezar de cero en la siguiente.

2.5 Suposiciones y dependencias

- **S-01**: Node.js 20.19+ o 22.12+ (probado con Node 24) para ejecutar el proyecto.
- **S-02**: el navegador permite `localStorage`.
- **S-03**: `productos.json` contiene 51 productos en 10 categorías, extraídos del catálogo entregado por el curso. El caso menciona 340 referencias en la tienda real.

---

3. Requisitos funcionales

Estado: ✅ implementado · 🔄 parcial · ⬜ pendiente · ⏸ diferido a etapa posterior

3.1 Navegación y estructura

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-01	Todas las vistas comparten encabezado, navegación y pie de página mediante un layout común	Checklist EP2 §B	P2	✅	`layouts/LayoutPrincipal.jsx`
RF-02	La navegación entre vistas se resuelve con React Router, sin recargar la página	Anexo 1	P2	✅	`App.jsx`, `main.jsx`
RF-03	El enlace de la vista actual se marca como activo	Diseño	P2	✅	`Header.jsx` (`NavLink`)
RF-04	Una ruta desconocida muestra una página de «no encontrado»	Diseño	P2	✅	`pages/NoEncontrado.jsx`
RF-05	En 360 px la navegación se colapsa en un menú hamburguesa	Caso §0.3	P2	✅	`Header.jsx` (`menuAbierto`)
RF-06	El encabezado muestra la cantidad de productos en el carrito	Anexo 1, fig. 1	P2	✅	`Header.jsx` (cuenta líneas distintas)
RF-07	El encabezado indica si hay sesión: «Ingresar» o «Panel» y «Salir»	Diseño	P2	✅	`Header.jsx`

3.2 Catálogo de productos

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-10	Mostrar el catálogo con imagen, marca, modelo, descripción y precio en CLP	Caso §4	P2	✅	`components/Item.jsx`, `ItemList.jsx`
RF-11	Mostrar la disponibilidad: en stock, últimas unidades o agotado	Caso §4	P2	✅	`utils/estadoStock.js`
RF-12	Filtrar por categoría, conservando el filtro en la URL (`?categoria=ID`)	Anexo 1, fig. 4	P2	✅	`pages/Catalogo.jsx`
RF-13	Mostrar la ficha de un producto individual con su ficha técnica	Anexo 1, fig. 3	P2	✅	`pages/DetalleProducto.jsx`
RF-14	Búsqueda de productos por texto	Anexo 1, fig. 1	P3	⏸	
RF-15	Vista de ofertas	Anexo 1, fig. 3	P3	⏸	Requiere precios de oferta en el modelo de datos

3.3 Carrito y pedido

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-20	Agregar productos al carrito desde el catálogo y desde la ficha	Caso §0.1	P2	✅	`Item.jsx`, `DetalleProducto.jsx`, `App.jsx`
RF-21	Modificar la cantidad de un producto y quitarlo del carrito	Anexo 1, fig. 5	P2	✅	`pages/Carrito.jsx`, `utils/carrito.js`
RF-22	Vaciar el carrito completo	Anexo 1, fig. 5	P2	✅	`Carrito.jsx`
RF-23	Mostrar subtotal por producto y total del carrito	Anexo 1, fig. 5	P2	✅	`calcularTotal`
RF-24	No permitir agregar un producto agotado ni superar el stock	Caso §3.3	P2	✅	`limitarCantidad`; botón deshabilitado con stock 0
RF-25	Confirmar el pedido con datos del cliente y dirección de entrega	Anexo 1, fig. 6	P2	✅	`pages/Checkout.jsx`
RF-26	Elegir entre despacho a domicilio y retiro en tienda	Caso §0.1	P2	✅	`Checkout.jsx`
RF-27	Mostrar el resultado de la compra: exitosa o con error, con su resumen	Anexo 1, fig. 7 y 8	P2	✅	`pages/ResultadoCompra.jsx` (pago simulado)
RF-28	Consultar el estado del pedido: en preparación, despachado, entregado	Caso §0.1	P3	⏸	
RF-29	Descontar el stock automáticamente al registrar una venta	Caso §4	P3	⏸	
RF-30	Conservar el carrito al recargar la página	Diseño	P3	⏸	Hoy vive en el estado de `App`
RF-31	Boleta en PDF o por correo	Anexo 1, fig. 7	P3	⏸	

3.4 Usuarios y acceso

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-35	Inicio de sesión con correo y contraseña	Caso §0.1	P2	✅	`pages/Login.jsx`, `authService.js`
RF-36	Registro público de clientes	Anexo 1, fig. 3	P2	🟪	`pages/Registro.jsx` (pendiente; `validarPassword` ya existe con pruebas)
RF-37	Las rutas de administración redirigen al inicio de sesión cuando no hay sesión	Caso §0.2	P2	✅	`components/RutaProtegida.jsx`
RF-38	Cerrar sesión	Caso §0.1	P2	✅	`Admin.jsx`, `Header.jsx`
RF-39	Restringir vistas según el rol del usuario	Caso §0.2	P3	⏸	`RutaProtegida` solo verifica que haya sesión

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-40	Autenticación real con JWT y contraseñas protegidas	Caso §0.1	P3	⏸	
RF-41	Historial de compras del cliente	Caso §0.2, Anexo 1 fig. 10	P3	⏸	
RF-42	Perfil del usuario	Anexo 1, fig. 9 y 10	P3	⏸	

3.5 Administración

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-50	Panel de administración con indicadores del catálogo	Anexo 1, fig. 9	P2	✅	`pages/Admin.jsx`: total, stock crítico, agotados
RF-51	Listar los productos del catálogo	Anexo 1, fig. 10	P2	✅	`pages/AdminProductos.jsx`
RF-52	Crear un producto, rechazando códigos repetidos	Anexo 1 (CRUD)	P2	✅	`AdminProductoForm.jsx`, `crearProducto`
RF-53	Editar un producto, incluido su stock	Anexo 1 (CRUD)	P2	✅	`AdminProductoForm.jsx`, `actualizarProducto`
RF-54	Eliminar un producto, con confirmación	Anexo 1 (CRUD)	P2	✅	`AdminProductos.jsx`, `eliminarProducto`
RF-55	Listar productos con stock crítico	Anexo 1, fig. 10	P2	✅	`Admin.jsx`, sección «Por reponer»
RF-56	Los cambios del CRUD persisten y se reflejan en la tienda	Anexo 1 (persistencia)	P2	✅	`productoService.js` sobre `localStorage`
RF-57	Gestión de categorías	Anexo 1, fig. 10	P3	⏸	
RF-58	Gestión de usuarios: crear, editar, desactivar y asignar roles	Caso §0.1, Anexo 1 fig. 10	P3	⏸	
RF-59	Órdenes y boletas, con cambio de estado	Caso §0.2, Anexo 1 fig. 10	P3	⏸	
RF-60	Reportes de ventas por período	Caso §4, Anexo 1 fig. 10	P3	⏸	

3.6 Contenido institucional

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-65	Página de inicio con destacados y accesos al catálogo y a las categorías	Anexo 1, fig. 1	P2	✅	`pages/Inicio.jsx`
RF-66	Página Nosotros con la presentación de la tienda	Anexo 1, fig. 3	P2	✅	`pages/Nosotros.jsx`

ID	Requisito	Origen	Etapa	Estado	Dónde
RF-67	Formulario de contacto con validaciones	Anexo 1, fig. 3	P2	✅	`pages/Contacto.jsx` (no envía a un servidor)
RF-68	Blog con publicaciones y su detalle	Anexo 1, fig. 3	P3	⏸	
RF-69	Mapa interactivo con la tienda y los puntos de retiro	Caso §0.1	P3	⏸	

3.7 Formularios y validaciones

Todos los formularios son **controlados**: cada campo vive en el estado del componente y la validación ocurre al enviar. Las reglas son funciones puras en `frontend/src/utils/validaciones.js`, cubiertas por 15 pruebas unitarias.

ID	Requisito	Formularios	Estado
RF-70	Los campos obligatorios no pueden enviarse vacíos	Contacto, Login, Checkout, Producto	✅
RF-71	El correo debe tener un formato válido	Contacto, Checkout	✅
RF-72	El precio debe ser un entero mayor que 0 y el stock un entero mayor o igual a 0	Producto	✅
RF-73	Con despacho a domicilio son obligatorias calle, región y comuna; las comunas dependen de la región	Checkout	✅
RF-74	La contraseña tiene al menos 6 caracteres y una mayúscula, y su repetición coincide	Registro	▫ (regla implementada y probada; falta el formulario)
RF-75	Cada error se muestra junto a su campo con un mensaje comprensible	Todos	✅ (`Campo`, `Selector`)

---

4. Requisitos no funcionales

4.1 Interfaz y diseño visual

RNF-01: diseño responsivo. La interfaz se adapta a 360, 768 y 1280 px con los prefijos de Tailwind CSS (`sm:`, `md:`, `lg:`), *mobile first*. La grilla del catálogo pasa de 1 a 2, 3 y 4 columnas; en móvil la navegación se colapsa en un menú hamburguesa; el checkout muestra el resumen primero; la tabla de administración se desplaza horizontalmente con la columna Código fija. **Estado:** ✅

RNF-02: sistema visual. Estética «latón»: marfil de fondo, laca oscura para el encabezado y latón como acento, definidos como tokens en `frontend/src/index.css` (`@theme`). Tipografía Work Sans para texto y DM Serif Display para títulos. **Estado:** ✅

Uso	Token	Color
Fondo de página	`marfil-100`	`#f5efe0`
Superficie de tarjetas	`marfil-50`	`#fbf7ee`
Texto principal y encabezado	`laca-900`	`#1a1714`
Texto secundario	`tinta-600`	`#5c5247`
Acento y botones	`laton-500`	`#c9a227`
Últimas unidades, errores	`cobre-600`	`#9a5424`
En stock, confirmaciones	`patina-600`	`#356b5e`

RNF-03: accesibilidad. Contraste WCAG 2.2 AA, foco visible, objetivos táctiles de 44 px, `alt` en imágenes, `aria-pressed` en los filtros, `aria-invalid` y `role="alert"` en los errores de formulario, `aria-expanded` en el menú móvil y animaciones desactivadas con `prefers-reduced-motion`. **Estado:** ✅

4.2 Calidad del código

- **RNF-10**: estructura por responsabilidad: `layouts/`, `pages/`, `components/`, `services/`, `data/`, `utils/`. **Estado:** ✅
- **RNF-11**: los componentes no leen `data/` directamente; todo acceso a datos pasa por `services/`. **Estado:** ✅
- **RNF-12**: el estado compartido se eleva al ancestro común más cercano (`App`) y se entrega por `props`; el estado que usa un solo componente queda en él. **Estado:** ✅
- **RNF-13**: ESLint sin errores (`npm run lint`). **Estado:** ✅
- **RNF-14**: lógica repetida extraída a módulos compartidos (`formatearPrecio`, `Selector`, `imagenProducto`). **Estado:** ✅

4.3 Pruebas

- **RNF-20**: entorno de pruebas con Vitest, `jsdom` y React Testing Library. **Estado:** ✅
- **RNF-21**: al menos 10 pruebas unitarias de lógica, validaciones, componentes e interacciones. **Estado:** ✅ **(78 pruebas en 15 archivos)**
- **RNF-22**: uso de mocks y spies (`vi.fn`, `vi.mock`, `vi.spyOn`, `vi.useFakeTimers`). **Estado:** ✅
- **RNF-23**: reporte de cobertura con `npm run test:coverage` y su análisis. **Estado:** ✅ **(75 % de líneas; utils 100 %, services 92,5 %)**. Ver `cobertura-testing-EP2.md`.

4.4 Seguridad

- **RNF-30**: en el Parcial 2 **no existe seguridad real**. La sesión se guarda en `localStorage` y puede modificarse manualmente; las contraseñas de los usuarios simulados están en texto plano en `usuarios.json`. La sesión guardada no incluye la contraseña.
- **RNF-31**: en el Parcial 3 se incorpora autenticación con JWT, control de acceso por rol y almacenamiento seguro de credenciales en el backend.

---

5. Modelo de datos y arquitectura

5.1 Datos simulados (`frontend/src/data/`)

Archivo	Contenido
`productos.json`	51 productos (catálogo inicial)
`categorias.json`	10 categorías
`usuarios.json`	3 usuarios de prueba, uno por rol
`regiones.json`	16 regiones con sus comunas principales

```
// productos.json
{
  "codigo":      "GA001",                // identificador único del producto
  "nombre":      "Guitarra Acústica Folk",
  "marca":       "Yamaha",
  "modelo":      "F310",
  "categoriaId": "GA",                   // referencia a categorias[].id
  "categoria":   "Guitarras Acústicas",
  "precio":      129990,                 // CLP, entero
  "stock":       8,
  "descripcion": "Tapa de abeto, aros y fondo de meranti. Ideal para iniciantes."
}
```

Categoría	ID	Productos
Guitarras Acústicas	GA	5
Guitarras Eléctricas	GE	5
Bajos Eléctricos	BA	3

Categoría	ID	Productos
Baterías	BT	5
Teclados y Pianos	TC	4
Amplificadores	AM	4
Micrófonos	MI	4
Pedales de Efectos	PE	6
Accesorios	AC	10
Estudio y Grabación	ES	5

5.2 Estado en el navegador

Dónde	Contenido	Declarado en
`localStorage` → `usuarioActual`	Sesión simulada: `{ correo, nombre, rol }`	`services/authService.js`
`localStorage` → `productos`	Copia mutable del catálogo, creada la primera vez desde `productos.json`	`services/productoService.js`
Estado de `App` (`useState`)	Carrito: `[{ codigo, nombre, precio, stock, cantidad }]`	`App.jsx`

5.3 Servicios

Función	Devuelve	Errores
`obtenerProductos(categoriaId?)`	Lista (filtrada si hay categoría)	—
`obtenerProductoPorCodigo(codigo)`	Producto o `null`	—
`obtenerCategorias()`	Lista de categorías	—
`crearProducto(producto)`	El producto creado	«Ya existe un producto con ese código»
`actualizarProducto(codigo, cambios)`	El producto actualizado	«No existe un producto con ese código»
`eliminarProducto(codigo)`	El producto eliminado	«No existe un producto con ese código»
`obtenerRegiones()`	Regiones con sus comunas	—
`iniciarSesion(correo, password)`	`true` / `false`	—
`obtenerSesion()`, `estaAutenticado()`, `cerrarSesion()`	Sesión, booleano, nada	—

Todas las de productos y regiones son `async` (devuelven promesas), igual que lo harán las llamadas a la API REST del Parcial 3.

5.4 Organización del código

```
frontend/src/
├── main.jsx                punto de entrada; envuelve la app en BrowserRouter
├── App.jsx                 tabla de rutas y estado compartido del carrito
├── index.css               Tailwind + tokens del sistema visual (@theme)
├── layouts/
│   └── LayoutPrincipal.jsx  Header + <Outlet /> + Footer
├── pages/                  una vista por ruta (14 páginas)
├── components/
│   ├── Header.jsx          marca, navegación, carrito, sesión y menú móvil
│   ├── Footer.jsx          pie de página
│   ├── Item.jsx            tarjeta de producto
│   ├── ItemList.jsx        grilla de productos
│   ├── Chip.jsx            botón de filtro
│   ├── Boton.jsx           botón o enlace con estilo de botón
│   ├── Campo.jsx           input/textarea con etiqueta y error
│   ├── Selector.jsx        select con etiqueta y error
│   ├── ResumenPedido.jsx   líneas y total de un pedido
│   └── RutaProtegida.jsx   redirige a /login si no hay sesión
├── services/
│   ├── productoService.js  catálogo, categorías y CRUD sobre localStorage
│   ├── authService.js      sesión simulada
│   └── regionService.js    regiones y comunas
├── data/                   archivos JSON simulados
├── utils/                  funciones puras: carrito, validaciones, estadoStock,
│                           formatearPrecio, imagenProducto
└── test/setup.js           configuración de Vitest
```

5.5 Rutas

Ruta	Vista	Acceso
`/`	Inicio	Público
`/catalogo`	Catálogo	Público
`/producto/:codigo`	Ficha de producto	Público
`/nosotros`	Nosotros	Público
`/contacto`	Contacto	Público

Ruta	Vista	Acceso
`/carrito`	Carrito	Público
`/checkout`	Confirmación del pedido	Público
`/compra/exito`, `/compra/error`	Resultado de la compra	Público
`/login`, `/registro`	Inicio de sesión y registro	Público
`/admin`	Panel de administración	Requiere sesión
`/admin/productos`	Listado de productos	Requiere sesión
`/admin/productos/nuevo`, `/admin/productos/:codigo`	Crear y editar producto	Requiere sesión
`*`	No encontrado	Público

El caso (§0.2) exige que ninguna vista sea accesible sin autenticación salvo inicio de sesión y registro. En el Parcial 2 el catálogo y la compra se mantienen públicos porque la autenticación es simulada; la regla se aplica en el Parcial 3 junto con el backend (RF-39, RF-40).

---

6. Decisiones de diseño registradas

ID	Decisión	Justificación
DEC-01	Tailwind CSS v4 en lugar de Bootstrap	Autorizado por el curso. Los prefijos de breakpoint expresan el comportamiento responsivo en cada componente, y la paleta propia vive en `@theme` sin sobrescribir un tema prearmado
DEC-02	Capa `services/` entre los componentes y los datos	En el Parcial 3 solo se reemplaza esa capa por `fetch` contra la API REST, sin tocar los componentes
DEC-03	Las funciones de servicio son `async` aunque lean datos locales	Los componentes ya consumen promesas, igual que lo harán con la API real
DEC-04	El filtro de categoría vive en la URL (`useSearchParams`)	Sobrevive a recargar la página y se puede compartir como enlace
DEC-05	El estado del carrito vive en `App.jsx`	Es el ancestro común del encabezado, el catálogo, la ficha, el carrito y el checkout
DEC-06	Un layout común con `<Outlet />`	El encabezado y el pie existen una sola vez
DEC-07	Validaciones y reglas del carrito como funciones puras en `utils/`	Se reutilizan entre formularios y se prueban sin renderizar componentes
DEC-08	La disponibilidad se calcula en `utils/estadoStock.js`	Una sola regla para «agotado» y «últimas unidades»

ID	Decisión	Justificación
DEC-09	Vitest + React Testing Library	Actualización técnica indicada por el curso; comparte configuración con Vite
DEC-10	Imágenes de producto con `import.meta.glob`	Vite las incluye en el build y resuelve cada una por el código del producto
DEC-11	CRUD sobre una copia del catálogo en `localStorage`	Los cambios del administrador persisten entre recargas y se ven en la tienda, sin backend
DEC-12	Un mismo formulario para crear y editar, según `useParams`	Una sola página y un solo conjunto de validaciones; al editar, el código queda bloqueado
DEC-13	El pedido viaja al resultado en el `state` de la navegación	La vista de resultado no necesita un almacén global; si se entra por URL, muestra «Sin pedido»
DEC-14	El pago es simulado, con una casilla para forzar el rechazo	Permite demostrar las dos vistas de resultado del Anexo 1 sin pasarela de pago
DEC-15	El contador del encabezado cuenta productos distintos	Es la convención más común en tiendas; la alternativa (unidades) es un `reduce`

---

7. Estado de avance del Parcial 2

Área	Avance
Estructura, layout y rutas	✅ completo
Catálogo y filtro por categoría	✅ completo
Ficha de producto	✅ completo
Carrito y pedido	✅ completo
Formularios y validaciones	✅ completo
Inicio y cierre de sesión	✅ completo
Registro de usuarios	⬜ pendiente (regla de contraseña lista)
Panel y CRUD de productos	✅ completo
Menú hamburguesa en 360 px	✅ completo
Sistema visual y accesibilidad	✅ completo
Pruebas unitarias	✅ 78 pruebas, todas pasan
Documento de cobertura del testing	✅ `cobertura-testing-EP2.md`
Vistas sugeridas no construidas (blog, ofertas, búsqueda, categorías, usuarios, órdenes, reportes, perfil)	⏸ diferidas

8. Ejecución del sistema

```
cd frontend
npm install
npm run dev
```

Comando	Qué hace
`npm run dev`	Servidor de desarrollo de Vite (http://localhost:5173)
`npm run build`	Build de producción en `frontend/dist/`
`npm test`	Ejecuta las pruebas unitarias una vez
`npm run test:coverage`	Pruebas con reporte de cobertura en `frontend/coverage/`
`npm run lint`	ESLint

Usuario administrador de prueba: el primero de `frontend/src/data/usuarios.json`.

9. Historial de versiones

Versión	Fecha	Etapa	Cambios
2.0 (borrador)	2026-10-12	Parcial 2	Especificación del frontend en React: requisitos por módulo, roles, rutas, modelo de datos simulado, estrategia de pruebas y decisiones de diseño
2.0	**2026-10-20**	**Parcial 2 (entregada)**	Estados actualizados a la versión entregada: carrito, checkout y resultado, sesión, panel y CRUD de productos, menú móvil. Se agregan las vistas sugeridas del Anexo 1 no construidas como diferidas, los servicios, el estado en el navegador y las decisiones DEC-11 a DEC-15. Pruebas: 78, cobertura 75 %